

Topology-Aware Polymorphism for Embedded Shading Languages

Ling Zhu*

Unity
Guangzhou, China
zhulingvision@gmail.com

Shaokun Zheng[†]

Huawei
Hangzhou, China
mango@live.cn

Liulei Zou*

Jiangsu University of Technology
Changzhou, China
graphzou@gmail.com

Kun Xu[†]

Tsinghua University
Beijing, China
xukun@tsinghua.edu.cn

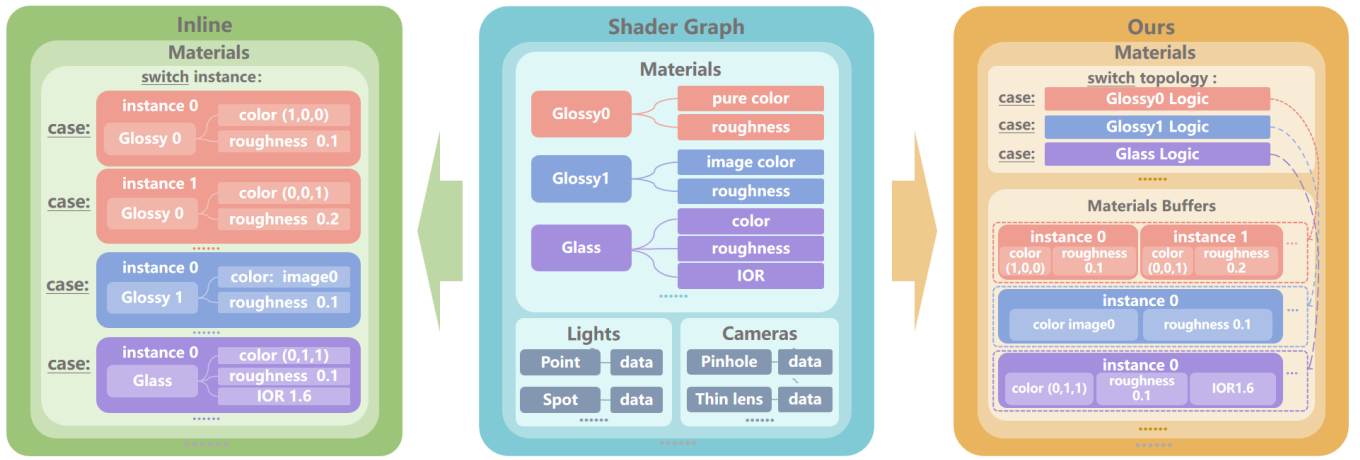


Figure 1: Naive ESLs (Inline) vs. Topology-Aware Polymorphism (Ours). Naive methods duplicate graphs and inline parameters for every instance. Our architecture decouples logic and data using a Topology-Dispatch Model (compiling logic once per shared graph topology) and a Parameter Aggregate Mechanism (managing instance-specific parameters in separate global buffers).

ACM Reference Format:

Ling Zhu, Liulei Zou, Shaokun Zheng, and Kun Xu. 2025. Topology-Aware Polymorphism for Embedded Shading Languages. In *SIGGRAPH Asia 2025 Posters (SA Posters '25)*, December 15–18, 2025, Hong Kong, Hong Kong. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3757374.3771513>

1 Introduction

Sophisticated rendering techniques for high-quality visual effects are often implemented with polymorphic shader graphs, whose topology is a dataflow structure, and instance is that structure with concrete parameters. Most production renderers use JIT compilation for these graphs, whereas Cycles uses dynamic interpretation.

*These authors contributed equally to this research.

[†]Joint corresponding authors.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SA Posters '25, Hong Kong, Hong Kong

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2134-2/25/12

<https://doi.org/10.1145/3757374.3771513>

Common game engines (like Unreal and Unity) prioritize performance by predefined string blocks over flexibility, except for a few high-level systems (like Blueprints, which use interpreters).

Domain-Specific Languages (DSLs) [He et al. 2018] and their subset Embedded Shading Languages (ESLs) [Zheng et al. 2022] offer an alternative. ESLs, embedded in a host language, enable unified host-device programming and host-context-aware code generation. A straightforward approach, as employed in LuisaRender [Zheng et al. 2022] and other similar ray-tracing frameworks, addresses polymorphism by generating code for each unique instance. As shown in Figure 1, this approach instance-specific parameters as literal values directly into the shader logic (in a switch-case). While it achieves high performance, the tight coupling of data and logic leads to the combinatorial explosion of specialized variants, severely limiting scalability in complex scenes. Moreover, the tight coupling necessitates full recompilation for any change and prevents hot reloading (on-the-fly shader reloading) [Hanna et al. 2025].

We propose a topology-aware architecture to address ESL polymorphism challenges by decoupling shader logic from instance data, as shown in Figure 1, thus reducing compilation and rendering time while also enabling practical hot reloading.



Figure 2: This comparison was performed using an AMD Ryzen 9 7950X CPU and an NVIDIA GeForce RTX 3070 Ti GPU. Image was rendered at 1920×1080 with 1000 samples per pixel (spp). The reported runtime is the average time of image (1 spp).

2 Proposed Topology-Aware Polymorphism

2.1 Topology-Dispatch Model

Our architecture compiles the shader logic once per topology, rather than for each individual instance, to mitigate the scalability bottleneck of large instance counts. A topology is strictly defined by the graph's structural layout (the types and connections of its nodes), which is independent of the specific values of the parameters. For example, as shown in Figure 1, two instances with different colors but an identical node structure (e.g., "Glossy0" instances) share the same topology. In contrast, two graphs with different node layouts (e.g., "Glossy0" vs. "Glossy1") are considered distinct topologies.

This approach establishes a fundamental separation of concerns: the compiled shader logic is reusable for all instances of a shared topology, while all instance-specific parameters are managed separately. This significantly reduces code volume and compilation overhead compared to naive methods that inline parameters and duplicate code for every instance. Furthermore, integrating topology-based dispatch logic minimizes branching in the final shader code, which can improve runtime performance.

2.2 Parameter Aggregate Mechanism

To enhance the flexibility and efficiency of runtime parameter updates, our approach aggregates data for all instances of a given shader graph topology (e.g., "Glass" material) into contiguous GPU buffers rather than inlining parameters for each instance. This allows for direct modification of instance-specific parameters at runtime via buffer offsets, eliminating the need for shader recompilation. For example, changing the color of a "Glossy" material instance only requires updating its data in the shared buffer.

This mechanism not only overcomes the limitations of parameter inlining but also improves runtime performance through enhanced data locality and reduced branch divergence on the GPU. Furthermore, it facilitates shader graph reuse across instances, which ensures that shared logic is efficiently executed by a shared logic node to avoid redundancy. For instance, if a texture is required by two properties of a material, the shared logic node reuses the texture sampling result instead of performing the sampling operation twice.

2.3 Hot Reload Mechanism

Any logic or parameter change in the naive approach forces full recompilation. Our architecture explicitly separates the compiled shader logic (handled per topology) from buffered instance parameter data.

This means that changing parameters only requires updating the parameter buffers, without needing shader recompilation.

Furthermore, this decoupling enables seamless integration with a module-level hot reloading system. Such a system compiles updated source code into new dynamic-link libraries, which the application loads at runtime to replace existing code modules, including updated shader logic. To ensure a smooth transition, the system handles new instance creation, old instance data transfer, reference updates, and static variable state management. By minimizing shader compilation costs, our architecture enables interactive shader development.

3 Experiments

We performed comparative experiments using our Corona Engine, which was inspired by LuisaRender [Zheng et al. 2022]. As shown in Figure 2, our architecture significantly reduces both rendering and compilation times across varying scene complexities. Even in the Cornell Box, we observed slight runtime speed improvements and a 50% increase in compilation efficiency. By reducing branching, our method reduces the compilation time significantly, boosts throughput, and enhances real-time performance. These advantages become more pronounced with an increasing number of instances.

4 Conclusion

By decoupling shader logic from instance data, our topology-aware polymorphism effectively mitigates the scalability and runtime flexibility issues of naive compiled ESLs. While we focus on ray tracing, this logic-data separation is universally applicable. It enables ESLs to be effectively applied to other pipelines (such as rasterization and compute), which mitigates the more severe adverse effects of the "naive inlining" problem and provides a more structured and efficient programming model for a wider range of graphics tasks.

References

- Carol Hanna, David Elliman, Wolfgang Emmerich, Federica Sarro, and Justyna Petke. 2025. Behind the Hot Fix: Demystifying Hot Fixing Industrial Practices at Zühlke and Beyond. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (Clarion Hotel Trondheim, Trondheim, Norway) (FSE Companion '25)*. Association for Computing Machinery, New York, NY, USA, 411–421. doi:10.1145/3696630.3728564
- Yong He, Kayvon Fatahalian, and Theresa Foley. 2018. Slang: language mechanisms for extensible real-time shading systems. *ACM Trans. Graph.* 37, 4, Article 141 (July 2018), 13 pages. doi:10.1145/3197517.3201380
- Shaokun Zheng, Zhiqian Zhou, Xin Chen, Difei Yan, Chuyan Zhang, Yuefeng Geng, Yan Gu, and Kun Xu. 2022. LuisaRender: A High-Performance Rendering Framework with Layered and Unified Interfaces on Stream Architectures. *ACM Trans. Graph.* 41, 6, Article 232 (Nov. 2022), 19 pages. doi:10.1145/3550454.3555463